Configuring the Database

The SQLite website is refreshingly honest when it comes to describing what this database is good at and what it's not good at.[84] In particular, SQLite isn't recommended for high-volume, high-concurrency websites with large datasets. And, of course, we want our website to be such a website. Plenty of alternatives to SQLite, both free and commercial, are available. We'll go with PostgreSQL.

Looking at the `docker-compose.yml` file, the first service listed is named `db`. This name serves as the host name for the container. Each service defined in the container is deployed on a private network isolated from the rest of the world except for ports that you decide to expose. Inside the network, services refer to each other by host name.

On the `db` host we run the stock postgres docker image[85] from Docker Hub.[86]

On top of that image, we mount a disk volume that we've named `pgdata`. This volume will be created as a file within the filesystem of the machine running the container. Placing the contents of the database outside of the container allows us to update the software image without affecting the data.

Finally, we define an environment variable containing the password we use to access the database. As this image has no external ports defined, and therefore no way to access it outside of the isolated environment that docker compose provides, this doesn't concern us for now. We may need to revisit once we deploy this application on a public cloud with a different configuration.

Now that we've defined the database container, we change the configuration of the production database from using the sqlite3 adapter to using the postgresql adapter:

rails7/depot_td/config/database.yml

```
# SQLite. Versions 3.8.0 and up are supported.
#   gem install sqlite3
#
```